POLITECNICO
MILANO 1863

AN2DL - First Challenge Report
Backprop Boys

Alessandro Zanoni, Jacopo Valentini, Arianna Balducci, Renato Gallicola

alessandrozanoni, jacopovalentini02, ariannabalducci1, renatogallicola01

277728, 279663, 302753, 252643

November 17, 2025

1 Introduction

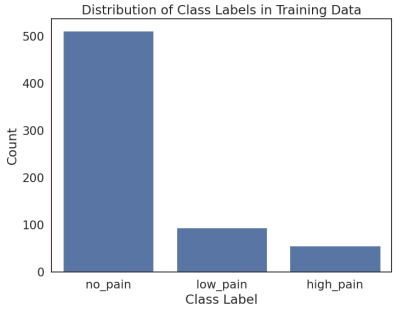
This project focuses on a *multivariate time series classification* task using modern *deep learning techniques* [5]. Our approach consisted of:

- Carefully analyzing the PiratePain dataset and its feature groups.
- Developing a baseline model architecture based on a **CNN–GRU** sequence encoder, leveraging gradient-based learning principles [6].
- Increasing complexity through ensembling, windowing strategies, augmentation. We additionally introduced a **binary classifier**, a **gating mechanism** and a model based on InceptionTime trained on a richer version of the dataset, all to improve the distinction between low and high pain.

2 Problem Analysis

In this challenge, **each sample** corresponds to a *multivariate time series* of **length 160**, describing the temporal evolution of a subject's motion and pain-related measurements.

- **Dataset characteristics**
  The features are grouped into heterogeneous families:
  1. *pain_survey* variables represent estimates of perceived pain.
  2. *n_legs*, *n_hands*, *n_eyes* encode **static subject attributes**.
  3. *joint_00–joint_30* are continuous joint-angle trajectories and constitute the majority of the input. The dataset shows a concrete variability in the dynamics of the joint features, which suggests that **temporal modeling** is essential.
- **Main Challenges**
  1. The **size** of the dataset is relatively small, this increases the risk of *overfitting*.
  2. The three classes show a moderate **imbalance**, especially due to the prevalence of the no_pain class. This could biases the majority of the naive models.



3. Pain-related information is not always evident from raw joint trajectories, making *feature extraction* crucial, although not manual but made by the convolutional modules.

3 Method

3.1 CNN+GRU Classifier

Each 160-step sequence was segmented into overlapping temporal windows. The proposed **CNN+GRU Classifier** is a compact hybrid model for multivariate time-series classification, composed of: **i) CNN feature extractor**: a small stack of Conv1D layers with BatchNorm, ReLU and Dropout, capturing local temporal patterns. **ii) GRU encoder**: a single-directional GRU processes the convolutional features, learning short- and mid-range temporal dependencies efficiently [1, 4]. **iii) Attention module**: assigns importance weights to the GRU hidden states to highlight informative time steps. **iii)Fully connected classifier**: the aggregated representation is fed into a small MLP producing logits for the three pain classes. Different window lengths and strides were used as temporal augmentation. During inference, **Test-Time Augmentation (TTA)** was applied by averaging predictions across multiple windows. Training relied on AdamW with weight decay, a ReduceLROnPlateau scheduler, class-weighted cross-entropy with label smoothing, and early stopping—improving optimization stability and mitigating class imbalance.

3.2 Ensembling of Different CNN–GRU Models

To improve robustness and reduce variance, we selected two main CNN–GRU models with different window sizes and strides, referred to as Model A and B. Several configurations were selected through Optuna-based hyperparameter search, and all models were trained using subject-wise StratifiedGroupKFold splits to prevent user leakage. Implementations were based on PyTorch [7]. User-level logits were obtained by averaging window predictions for each subject. We explored several ensemble strategies: **Simple logit averaging**: arithmetic mean of logits from two strong models. **Weighted averaging**: manually tuned weights $(\alpha,\ 1-\alpha)$. **CMA-ES optimized ensemble**: the Covariance Matrix Adaptation Evolution Strategy [3] was used to automatically optimize ensemble weights by maximizing validation F1-micro. Optimization was performed on out-of-fold (OOF) validation logits, allowing evaluation on unseen data without requiring a separate validation model. The CMA-ES ensemble consistently outperformed simpler combinations, indicating that the individual models capture complementary temporal features.

3.3 Binary Refinement and InceptionTime Integration

To further discriminate between *low_pain* and *high_pain*, we added a dedicated binary predictor and a separate InceptionTime-based model.
Binary CNN–GRU classifier: a compact binary model, structurally aligned with the main classifier, limited to the {low, high} decision. **InceptionTime augmentation**: an InceptionTime block [2] , enriches temporal representations through multi-scale convolutions, complementing the single-scale CNN frontend. **Gating mechanism**: the binary classifier, the InceptionTime branch, and the main ensemble are combined through a confidence-based gate. When the ensemble is confident about *no_pain*, its prediction is retained; otherwise, uncertain {low, high} decisions are refined by fusing ensemble and binary logits through weighted averaging, reducing many low/high misclassifications.

4 Failed Experiments

During the course of the challenge we tried a huge number of different model architectures, optimization and cleaning techinques, here a brief summary:

- **LSTM, BiLSTM, CNN+LSTM**
  Recurrent networks such as LSTMs, especially in their bidirectional form, are standard tools for capturing long-term temporal dependencies. We expected them to extract richer temporal patterns. However, LSTMs tend to overfit quickly on medium-sized datasets like ours, especially when trained from scratch.The temporal patterns in PiratePain appear to be local and short-range which reduces the benefit of LSTMs. Overall, CNN+GRU turned out to be simpler, more stable, and more robust to noise.

- **Ensembling of 5-fold CV trained models with three no-split trained models**
  The hypothesis was that CV models would capture different decision boundaries while no-split models trained on all data would reduce variance. Therefore, combining them would reduce both overfitting and underfitting. Anyway this could be failed since the models may not have been different enough, in fact, similar architectures trained on similar data tend to produce highly correlated errors. Moreover some folds likely produced unstable or noisy logits, especially given the relatively small dataset.

- **Implementing focal loss function on different architectures**
  Since Focal loss is effective when the dataset is strongly unbalanced, we tried to implement this function to penalize mistakes made by the models in not recognizing the not so well represented classes like *high_pain* and *low_pain*. But in the end, we used **Cross entropy loss** with a good label smoothing rate since Focal loss tends to destabilize optimization on small datasets: it amplifies noise and can lead the model to chase outliers.

5 Results and Discussion

Table 1 summarizes the performance of the main architectures evaluated in this work. Across all experiments, the CNN–GRU baseline provided a solid starting point, confirming that a combination of local convolutional features and recurrent temporal modeling is well suited for this dataset. However, its performance was limited by the moderate dataset size and the variability of the joint trajectories. Introducing window-based processing and Test-Time Augmentation (TTA) improved the stability of the predictions, but the most significant gain came from ensembling multiple CNN–GRU models. In particular, the CMA-ES–optimized ensemble achieved the best trade-off between validation loss and F1-micro, showing that different models capture complementary temporal cues. Finally, the integration of the binary refinement module and the lightweight InceptionTime branch further improved the separation between *low_pain* and *high_pain*, leading to the highest Kaggle leaderboard score among the tested approaches. Overall, these results highlight the effectiveness of combining recurrent architectures, multi-scale temporal modeling, and evolutionary ensemble optimization for robust time-series classification under noisy and data-limited conditions.

6 Conclusions

Future work may focus on improving interpretability and designing lighter architectures that preserve accuracy while reducing computational complexity. Here the author's contributions:

- **Alessandro Zanoni**: analyzed accurately the dataset, found correlation among features. Tried LSTM, BiGRU, focal loss function, different kind of models ensembling and cross validations techniques.

- **Jacopo Valentini**: implemented the main CNN–GRU architecture, designed the training pipeline and performed extensive hyperparameter tuning using Optuna. He also developed the CMA-ES optimized ensemble.

- **Arianna Balducci**: developed the binary CNN–GRU classifier for refining low/high pain predictions, integrated the InceptionTime module for multi-scale temporal feature extraction.

- **Renato Gallicola**: implemented the windowing and sequence-generation pipeline, handled user-level logits aggregation, and experimented different ensemble strategies.

Table 1: Comparison of the main evaluated architectures on the validation set and Kaggle leaderboard.

Architecture	**Val. loss**	**Val. F1-micro**	**Kaggle score**
CNN–GRU baseline	0.3412	0.9306	0.9224
CMA–ES ensemble (A+B)	0.1848	0.9614	0.9476
Ensemble + binary gate + InceptionTime	0.1971	0.9575	0.9603

References

[1] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using rnn encoder–decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.

[2] H. I. Fawaz, B. Lucas, G. Forestier, C. Pelletier, D. F. Schmidt, J. Weber, G. I. Webb, L. Idoumghar, P.-A. Muller, and P. Forestier. Inceptiontime: Finding alexnet for time series classification. *Data Mining and Knowledge Discovery*, 34(6):1936–1962, 2020.

[3] N. Hansen. The cma evolution strategy: A tutorial. In *Proceedings of the 14th International Conference on Parallel Problem Solving from Nature*, PPSN XIV, pages 835–868. Springer, 2016.

[4] A. Karpathy. The unreasonable effectiveness of recurrent neural networks, 2015. Blog post.

[5] Y. LeCun, Y. Bengio, and G. Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.

[6] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[7] PyTorch Team. Pytorch documentation. `https://pytorch.org/docs/stable/`. Accessed: 2025-11-17.